第13课 函数入门

一、课程信息

项目	内容
课次	第 13 课 / 共 40 课
阶段	阶段二：基础算法（本阶段第 1 课）
主题	函数入门 —— 给程序请个"小帮手"
前置知识	输入输出、变量与类型、运算符、分支、循环、一维数组（第 1-12 课）
课时	100 分钟

教学目标（可检验）

1. **能写出**：学生能独立定义一个带参数、有返回值的函数，并在 `main` 中正确调用它。
2. **能区分**：学生能准确说出"参数"和"返回值"分别是什么，并能区分 `void` 函数与有返回值函数的使用场景。
3. **能改造**：给出一段含有重复代码的程序，学生能将重复部分提取为函数，完成代码重构。

二、教学重点与难点

重点

- 函数的完整语法结构：`返回类型 函数名(参数列表) { 函数体; return 值; }`
- 函数的调用过程：传参 执行 返回
- `void` 函数与有返回值函数（`int`、`double`）的区别

难点

- **参数的"副本"机制**（值传递）：参数传进函数后，函数里改了不影响外面。小学生容易以为"传进去就变了"。
- **`return` 的含义**：既是"交出结果"，又是"函数到此结束"。
- **函数定义的位置**：必须写在 `main` 前面（本课阶段只讲这一种方式，暂不讲函数声明）。

三、课堂时间分配（100 分钟）

时段	时长	内容	形式
1	5 min	开场故事 + 引入"为什么需要函数"	讲授（互动提问）
2	15 min	函数概念讲解：榨汁机比喻 语法结构 第一个函数	讲授 + 板书演示
3	10 min	例题1 + 例题2 带练	师生共写代码
4	10 min	**练习1**：学生动手写一个函数	学生练习（教师巡视）
5	5 min	休息 + 成就展示（找写好的同学上来讲）	休息 / 互动
6	15 min	void 函数讲解 + 值传递"抄作业"比喻 + 例题3	讲授 + 演示
7	10 min	**练习2**：学生动手	学生练习
8	15 min	例题4（综合）+ 常见错误排雷	讲授 + 互动
9	10 min	**练习3 / 练习4**：综合练习	学生练习
10	5 min	课堂小结 + 布置作业 + 下节课预告	讲授

> 讲解段最长不超过 20 分钟，中间穿插提问和"举手抢答"。

四、讲稿提纲

4.1 开场故事（5 分钟）

> 同学们，想象一下——你开了一家"果汁店"。每来一个顾客，你都要：洗水果 切水果 放进榨汁机 倒进杯子 递给顾客。
>
> 如果来了 100 个顾客，你总不能把这五步写 100 遍吧？
>
> 所以我们买了一台**榨汁机**！只要把水果放进去（这就是**参数**），按一下按钮（这就是**调用**），它就会自动帮你做好果汁吐出来（这就是**返回值**）。
>
> 今天，我们要学的"函数"，就是程序里的**榨汁机**——一个你可以反复使用的**小帮手**！

互动提问：
- "如果我放进去苹果，出来的是什么？"——苹果汁（不同的参数，不同的结果）
- "如果我不需要它吐出果汁，只需要它帮我把水果切好呢？"——引出 `void`（只做事，不给东西回来）

4.2 函数的语法结构（15 分钟）

板书/PPT 展示函数模板：

```
返回类型 函数名(参数) {
    做的事情;
     return 结果;
}
```

用榨汁机对照：

函数组成	榨汁机对照	举例
返回类型	出来的是什么类型的产品	`int`（整数果汁）、`double`（小数果汁）、`void`（不出东西）
函数名	这台机器叫什么名字	`zhajizhi`、`add`、`maxVal`
参数	你放进去的材料	`int a, int b`（两个整数水果）
return	把产品递出来	`return a + b;`

关键口诀（带学生念一遍）：

> 先写"出什么"，再写"叫什么"，括号里写"吃什么"，大括号里写"做什么"，最后 `return` "吐什么"。

第一个函数演示（边讲边写）：

```
#include <iostream>
using namespace std;

// 这是我们的"加法小帮手"
int add(int a, int b) {
    int sum = a + b;
    return sum;
}

int main() {
    int result = add(3, 5);
    cout << result << endl;  // 输出 8
    return 0;
}
```

讲解要点：
- `add` 写在 `main` 上面——"帮手要先招聘好，才能在工作中叫他"
- `add(3, 5)` 就是"叫小帮手来干活"，3 和 5 是给他的材料
- `return sum` 就是小帮手把结果递给你
- `int result = add(3, 5)` —— 你用一个变量接住他递回来的结果

4.3 void 函数 + 值传递（15 分钟）

void 函数讲解：

> 有些小帮手，你叫他做事，他做完就行了，不需要递东西给你。比如你让一个小帮手"去黑板上写字"，他写完就好了，不用递什么东西回来。这种小帮手的类型就是 `void`——"什么都不返回"。

值传递——"抄作业"比喻：

> 当你调用函数的时候，参数就像是"抄了一份作业给小帮手"。小帮手拿到的是一份**副本**，他在副本上怎么涂改，都不会影响你原来的那份作业。

```
#include <iostream>
using namespace std;

void tryToChange(int x) {
    x = 100;  // 小帮手在自己的副本上写了100
    cout << "函数里面: x = " << x << endl;
}

int main() {
    int a = 1;
    tryToChange(a);
    cout << "函数外面: a = " << a << endl;  // a 还是 1!
    return 0;
}
```

> 看到了吗？小帮手把他的副本改成了 100，但你原来的 `a` 还是 1。这就叫**值传递**——传的是值的副本，不是原件！

4.4 课堂小结口诀（5 分钟）

带学生一起念：

> **函数四要素，记牢不会错：**
> **返回写前面，名字自己取，**
> **括号装参数，大括号写过程，**
> **return 交结果，void 不用还。**
> **调用写名字，括号填材料，**
> **帮手定义在前，main 里面才能用！**

七、常见错误与应对

错误 1：函数定义写在 main 后面

```
int main() {
    cout << add(3, 5);  //  编译器说"add 是什么？我不认识！"
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

应对：告诉学生——"帮手要先招聘，才能干活。写在 `main` 上面！"本课阶段统一要求函数定义写在 `main` 之前，后续再学函数声明（前置声明）。

错误 2：忘记写 return

```
int add(int a, int b) {
    int sum = a + b;
    // 忘记写 return sum; 了！
}
```

应对：编译器可能警告也可能不警告，但结果是未定义的（输出乱七八糟的数）。提醒学生——"帮手做完事忘了把结果交给你，你收到的就是一堆乱码！**有返回类型的函数，一定要写 return！**"

错误 3：调用时重新写了类型

```
int main() {
    int result = add(int a, int b);  //  调用的时候不要写类型！
}
```

应对：区分"定义"和"调用"——"招聘的时候要写简历（类型 + 名字），叫他干活的时候直接喊名字就行了！"
- 定义：`int add(int a, int b)`
- 调用：`add(3, 5)`、`add(x, y)`

错误 4：void 函数里写了 return 值

```
void sayHello() {
    cout << "Hello!" << endl;
    return 1;  //  void 函数不能 return 一个值!
}
```

应对："`void` 的意思是'空的'——这个帮手不给你递东西。你让他 `return 1`，他一脸懵——'你说好不用我递东西的呀！'"

错误 5：以为函数里改了参数，外面也会变

```
void changeIt(int x) {
    x = 999;
}

int main() {
    int a = 1;
    changeIt(a);
    cout << a << endl;  // 学生期望输出 999，实际输出 1
}
```

应对：重复"抄作业"比喻——"你把作业抄了一份给帮手，帮手在副本上写了 999，你手上的原件当然不变啊！"

九、下节课预告

> **第 14 课：二维数组**
> 同学们，我们之前学的数组是"一排格子"，下节课我们要学"一张表格"——**二维数组**！想象一个棋盘、一张座位表，都是二维数组。学了二维数组，我们就能处理更复杂的数据了！
> 预习小任务：想一想，生活中有哪些东西是"表格形状"的？（棋盘？教室座位？日历？Excel 表格？）下节课我们来聊聊！

> **教案设计说明**：本课是"阶段二：基础算法"的开篇，函数是后续所有算法学习的基础工具。教学策略以"榨汁机""小帮手""抄作业"三个核心比喻贯穿全课，帮助零基础小学生建立直觉。练习设计从模仿（练习1-2）到应用（练习3）到重构（练习4），逐步提升思维层次。课后作业选择洛谷基础题，确保学生能独立完成，建立信心。